CONTENTS

Advanced | AI agent architecture | Condensed | IBM Technology

The Four Types of Agent Memory: Working, Semantic, Procedural, Episodic

Know which store to reach for, what each costs you in context, and why forgetting is the hard part.

For engineers building agents who have to decide what the system remembers, and where · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

Watch the source video | Read the full lesson | Read the highlights

BEFORE YOU START

- What a context window is and that it is finite
- Basic familiarity with agents calling tools
- Comfort reading Python

Overview

A language model is stateless. Every capability that looks like memory is something the surrounding system built, and the useful question is not whether your agent has memory but which kind, held where, loaded when.

The CoALA framework — Cognitive Architectures for Language Agents, from a Princeton research team — names four types, and they map cleanly onto how people remember. Working memory is what is active right now. Semantic memory is what you know. Procedural memory is what you know how to do. Episodic memory is what happened to you before.

The distinction is worth more than a taxonomy, because the four differ in exactly the ways that matter to an implementation: whether they survive the session, whether they cost context tokens continuously or only on demand, who writes to them, and how they go stale. Working memory is volatile and always loaded. Semantic memory is stable and usually always loaded. Procedural memory is loaded on demand. Episodic memory accumulates by itself, and that is the one that turns into an engineering problem.

KEY TAKEAWAYS

- ✓ Models are stateless — every kind of agent memory is machinery the surrounding system provides.
- ✓ CoALA names four types: working, semantic, procedural and episodic, mirroring human memory.
- ✓ Working memory is the context window: fast, always present, volatile, and capped in size.
- ✓ Semantic memory is what the agent knows in general, and in production it is often just Markdown files rather than a vector database.
- ✓ Procedural memory is how the agent does things, packaged as skills with step-by-step instructions.
- ✓ Progressive disclosure keeps procedural memory affordable: advertise a short index, load the full instructions only on a match.
- ✓ Episodic memory is distilled experience, not saved transcripts — the compression is what makes it useful.
- ✓ Forgetting is the hardest part, because staleness and obsolescence have to be decided in code rather than felt.

01 Human memory as the map

0:00

> Four kinds of remembering that people do without noticing, which turn out to be four separate engineering problems.

KEY POINTS

- → Human memory splits into short-term, factual knowledge, learned skills and personal experience.
- → The four are retrieved differently, persist differently and fail differently.
- → Knowing a fact and being able to perform a skill are separate capabilities.
- → Agents need the same four, and each maps to different machinery.
- → Treating memory as one feature produces either amnesia or context exhaustion.

Four kinds of remembering, four kinds of failure

The right-hand column is what makes this a useful taxonomy rather than a tidy one: each type breaks in its own way, so each needs its own fix.

human	agent	fails by
what is on my mind now	working memory	overflowing, or vanishing
facts I know	semantic memory	going out of date
things I can do	procedural memory	never being loaded
what happened to me	episodic memory	piling up and going stale

02 CoALA: naming the four

1:25

> A framework from a Princeton research team that maps out the distinct memory types an agent needs.

KEY POINTS

- → CoALA identifies four memory types for language agents.
- → Four questions separate them: lifetime, writer, load moment, failure mode.
- → Working memory is written by the runtime; semantic and procedural by humans.
- → Episodic memory is the only type the agent writes for itself.
- → That self-writing property is both why it can improve and why it degrades.

The four, on the properties that decide the design

Read the columns rather than the rows. 'When it enters the prompt' predicts your token bill; 'who writes it' predicts who is responsible when it is wrong.

	lifetime	written by	enters prompt	fails by
working	session	runtime	always	overflow
semantic	permanent	humans	session start	drift
procedural	permanent	humans	on demand	not being found
episodic	permanent	the agent	by relevance	accumulation

03 Working memory: the context window

2:09

> Everything the agent can see right now — the conversation, the system instructions, whatever has been loaded into the prompt.

KEY POINTS

- → Working memory is the context window: conversation, instructions, loaded data.
- → Like RAM — fast because there is no retrieval, volatile, and capped.
- → Million-token windows still have a ceiling, and buried content gets lost before it is reached.
- → Usable capacity is lower than advertised capacity, by an amount you have to measure.
- → Context is a budget to allocate, not a container to fill.
- → Every chatbot has working memory; it is not what makes something an agent.

Where the budget actually goes

Worth computing for your own agent before adding anything to the prompt. Teams are routinely surprised by how much is spent before the user has said a word — and the fixed costs are paid on every single turn.

```
def budget(window=200_000):
    fixed = {
        "system prompt":        1_200,
        "tool schemas":         3_400,   # grows with every tool you add
        "semantic (CLAUDE.md)": 2_800,   # loaded at session start
        "skill index":            900,   # ~100 tokens x 9 skills
    }
    spent = sum(fixed.values())
    for name, n in fixed.items():
        print(f"  {name:<22} {n:>7,}")
    print(f"  {'--> left for work':<22} {window - spent:>7,} "
          f"({(window - spent) / window:.0%})")

# every turn pays the fixed cost again — it is not amortised
```

04 Semantic memory: what the agent knows

3:33

> The knowledge base — facts, rules, conventions, documentation. In production it is

frequently just Markdown.

KEY POINTS

- → Semantic memory holds facts, rules, conventions and documentation.
- → Vector databases and knowledge graphs are real implementations, and often unnecessary.
- → In production it is frequently a Markdown file loaded at session start.
- → Files are auditable, diffable and version-controlled with the code they describe.
- → Files are loaded whole, so they compete with the context budget and cap out in the low thousands of tokens.
- → Small, stable and always relevant → a file. Large or occasionally relevant → retrieval.

What belongs in a semantic memory file

Note how much of it is prohibitions. The mistakes an agent repeats are more expensive than the facts it lacks, and 'what not to do' is the part nobody thinks to write down.

```
# CLAUDE.md

## Architecture
Monorepo. `api/` is FastAPI, `web/` is Next.js, shared types in `packages/`.

## Commands
build:  make build          (never `npm run build` directly)
test:   make test           (runs unit + contract)
deploy: make ship           (staging only; production is manual)

## Conventions
- All timestamps stored UTC, rendered Europe/Madrid.
- Money is integer cents. Never float.
- Migrations are forward-only.

## Do not
- Do not edit `packages/types/generated/` — it is code-generated.
- Do not add dependencies without asking.
- Do not touch `legacy/` unless the task names it.
```

05 Procedural memory: how the agent does things

4:59

> Skills — a folder with a Markdown file describing what the skill does and the steps to perform it.

KEY POINTS

- → Procedural memory is instructions for performing a task, packaged as a skill.
- → A skill is a folder with a SKILL.md describing what it does and the steps to do it.
- → Semantic memory is declarative; procedural memory is imperative.
- → An agent can hold every relevant fact and still perform the steps wrong.
- → Test: 'what is the case?' is semantic, 'how, in what order?' is procedural.
- → Skills are reviewable files, portable across models, and fixed by editing.

A skill folder

The frontmatter is not documentation — the description is the only thing the agent sees until the skill is loaded, so it is what decides whether the skill is ever used at all. The supporting files stay on disk until execution needs them.

```
skills/release/
├── SKILL.md            what it does + the steps
├── checklist.md        pulled in during execution
└── rollback.sh         pulled in only if a step fails

# SKILL.md
---
name: release
description: Cut and publish a release. Use when asked to ship, tag,
  publish or roll back a version.
---

1. Confirm main is green in CI. Stop if not.
2. Bump the version in `pyproject.toml`. Nothing else.
3. Update CHANGELOG.md from commits since the last tag.
4. Tag `v<version>` and push the tag, not the branch.
5. Watch the publish job. On failure run `rollback.sh <version>`.
```

06 Progressive disclosure

5:41

> The agent advertises a lightweight index, loads full instructions on a match, and pulls in supporting files only during execution.

KEY POINTS

- → Loading every skill at once would exhaust the context budget before work began.
- → The agent sees only a name and description per skill — roughly a hundred tokens each.
- → Full instructions load when a task matches; referenced files load during execution.
- → Fifty skills cost about one skill's worth of context until one is actually used.
- → The description is the only visible part at selection time, so a vague one makes the skill invisible.
- → Semantic memory is always present; procedural memory is deliberately not.

Three stages, three costs

Run the numbers for your own skill set. The point is that stage one scales with how many skills you have and stages two and three scale with what the current task needs — which is the only reason a large skill library is possible at all.

```
stage 1  ADVERTISE   every turn
  release   : "Cut and publish a release. Use when asked to ship..."
  code-review: "Run a structured review. Use when asked to review..."
  ... 48 more                                          ~100 tokens each
                                                       ──────────────────
                                                        ~5,000 total

stage 2  LOAD        only on a match
  the full text of skills/release/SKILL.md            ~1,200 tokens

stage 3  EXECUTE     only when reached
  checklist.md when step 3 runs                         ~800 tokens
  rollback.sh  only if a step fails                          0 tokens

# 50 skills fully loaded would be ~60,000. this is ~7,000.
```

07 Episodic memory: what happened before

6:25

> The record of past interactions and decisions, and what was learned from them — distilled rather than stored whole.

KEY POINTS

- → Episodic memory records past interactions, decisions and what came of them.
- → Saving whole transcripts qualifies technically and works badly.
- → Production systems distil: keep the conclusion, discard the narrative.
- → A good note is one sentence, durable, and would change future behaviour.
- → This is the only type that improves without anyone editing a file.
- → It is also the only type nobody reviews before it is written.

Distillation, as a step the agent actually runs

The three conditions are what stop the store filling with restated context. Most sessions should produce nothing — an agent that writes a note every time has not been given a high enough bar.

```
DISTIL = """Review this session. Write at most 3 notes worth keeping.

Keep a note only if all three hold:
  - it would change what you do in a future session
  - it is still true tomorrow (not "the build is broken right now")
  - it is not already in CLAUDE.md or a skill

One sentence each. State the conclusion, not the story.
Return an empty list if nothing qualifies."""

def close_session(transcript, store):
    for note in model.extract(DISTIL, transcript, schema=Note):
        store.write(note.text, tags=note.tags, ts=now())

# most sessions should produce zero notes
```

08 Forgetting is an engineering problem

7:52

> What do you delete? When does something become obsolete? People forget

effortlessly; agents have to be told.

KEY POINTS

- → Deletion, not writing, is the hard part of episodic memory.
- → A stale note is worse than a missing one — it displaces correct behaviour.
- → Time-based decay handles relevance that fades.
- → Contradiction handling replaces superseded notes rather than storing both.
- → Scoped invalidation drops a class of notes when its context ends.
- → A human-readable store makes retention a five-minute review instead of a policy.

Three mechanisms that compose

None is sufficient alone. Decay misses notes that are wrong immediately; contradiction handling misses notes nothing contradicts; scoped invalidation misses everything not tied to a scope.

```
def maintain(store, now):
    # 1. decay — relevance fades, and unused notes fade fastest
    for note in store.all():
        age_days = (now - note.last_used).days
        if age_days > 90 and note.use_count == 0:
            store.drop(note)

    # 2. contradiction — replace, never keep both
    for note in store.all():
        newer = store.contradicting(note, after=note.ts)
        if newer:
            store.drop(note)          # retrieval must not have to choose

    # 3. scoped invalidation — the context itself ended
    for scope in store.dead_scopes():   # archived repo, closed account
        store.drop_where(scope=scope)
```

09 Not every agent needs all four

8:32

Match the memory architecture to what the agent actually does. Most need fewer types

than the taxonomy suggests.

KEY POINTS

- A reflex agent needs working memory only.
- A narrow task agent needs working plus procedural.
- A coding agent typically needs all four.
- Add a memory type only when you can name the failure it fixes.
- Unneeded memory costs context, latency and maintenance.
- Memory is what separates a chatbot's response from an agent's.

Three agents, three architectures

Notice the middle row. Skipping episodic memory there is a decision, not an omission — a password-reset agent that remembers users has taken on a data-retention problem for no gain.

	working	semantic	procedural	episodic
thermostat / router	●	·	·	·
password-reset agent	●	·	●	·
coding agent	●	●	●	●

Glossary

CoALA

Cognitive Architectures for Language Agents, a framework from a Princeton research team that identifies four memory types agents need.

Working memory

The context window: the conversation, instructions and loaded data the agent can see right now. Fast, volatile and capped.

Semantic memory

What the agent knows in general — facts, rules, conventions and documentation. Often a Markdown file loaded at session start.

Procedural memory

How the agent performs a task: ordered, step-by-step instructions, packaged as a skill.

Episodic memory

The record of past sessions and what was learned from them, kept as distilled notes rather than whole transcripts.

Context window

The maximum text a model can process in one call. It bounds working memory and everything loaded into it.

Progressive disclosure

Advertising skills by a short description, loading full instructions only on a match, and supporting files only during execution.

Skill index

The lightweight list of skill names and descriptions the agent always sees — roughly a hundred tokens per skill.

SKILL.md

The Markdown file defining a skill: what it does, when to use it, and the steps to perform it.

CLAUDE.md

A project-root Markdown file holding architecture, conventions, commands and prohibitions, loaded at the start of every session.

Distillation

Compressing a session into a few durable conclusions worth remembering, instead of storing the transcript.

Scoped invalidation

Dropping a whole class of memories when the context they belonged to ends — a project archived, an account closed.

Reflex agent

An agent that maps an input directly to an action with no persistence, needing only working memory.

Stale note

A stored memory that has become false. Worse than no memory, because it displaces correct behaviour rather than merely omitting it.

Check yourself

Answer first, then open each one.

An agent is told a project's deploy command mid-session and follows it correctly, then gets it wrong the next day. Which memory type is missing, and why is this not a model failure?

Semantic memory. The correction lived in working memory, which is volatile and ends with the session, and no write path existed to persist it. The model behaved correctly on both days given what it could see — the missing piece is a file loaded at session start, not a better model.

Why does progressive disclosure make a library of fifty skills affordable when loading them all would not be?

Because the always-present cost is only the index — a name and description per skill, roughly a hundred tokens each — while full instructions load only when a task matches and supporting files only when execution reaches them. Fifty skills cost around five thousand tokens instead of sixty thousand, so the context cost scales with what the current task needs rather than with how many skills exist.

A skill has perfect instructions and is never used. What is the most likely cause?

Its description. That is the only part visible at selection time, so if it does not name the situation and the words a user would actually say, the agent has no basis for loading it. The instructions are never read because the decision to read them happens before they are visible.

Why do production systems distil episodic memory rather than saving conversation transcripts?

A transcript is mostly wrong turns, restated context and dead ends, so retrieving it returns long passages of doubtful relevance at high token cost. A distilled note keeps the durable conclusion and discards the narrative, which is both far smaller and far more likely to change what the agent does next.

Why is a stale episodic note worse than having no episodic memory at all?

An empty store makes the agent ask, which is mildly inconvenient. A stale note makes it act confidently on information that has become false, displacing the correct behaviour rather than merely failing to supply it. This asymmetry is the argument for deleting aggressively when in doubt.

A password-reset support agent is given episodic memory so it can recall individual users. What has that decision actually bought and cost?

It has bought nothing the task needs — each reset is self-contained and the procedure comes from procedural memory. It has cost a data-retention obligation, a staleness problem and per-turn retrieval latency. Memory types should be added only when you can name the failure they fix.

Where to go next

- → Compute the fixed context cost of your own agent — system prompt, tool schemas, semantic file, skill index — and see what fraction of the window is gone before the user has typed anything.
- → Read the CoALA paper (Sumers, Yao, Narasimhan and Griffiths, 2023) for the fuller treatment, including how the memory types connect to the agent's decision procedure.
- → Take an agent you already run and classify everything currently in its system prompt as semantic, procedural or neither; the procedural parts usually belong in skills that are not always loaded.
- → Write the distillation step for a session you actually had, then check a week later how many of the notes are still true — the survival rate tells you whether your bar is high enough.
- → Instrument episodic retrieval to record which notes were retrieved and whether they were used, then delete anything never used after 90 days and see if anything breaks.
- → Rewrite the descriptions of skills that never trigger as explicit trigger conditions naming user phrasings, and measure how often they are selected afterwards.

→ Try the reverse experiment: strip semantic memory from a coding agent for a day and note how many corrections you repeat — that count is what the file is worth.
